



Yunnan University Software College

Professional Training Report

School of Software, Yunnan University

Personal Grades

(Students fill in (except the last column))

Serial number	Student number	Name	Major	Score
1	20243120102	ABBAS SYED MUHAMMAD ZAFAR	AI	
2				
3				

Semester: Autumn 2025

Course Name: Professional Training

Teacher: Ahmed zahir

Project Name: Sentiment Analysis with
DistilBERT & Chrome DB Integration

Team Leader:

Report completion Date: 11th December 2025

Content

1. Abstract	3
2. Introduction	3
2.1 Overview and Motivation	3
2.2 Objectives	4
3. Literature Review	4
3.1 Evolution of NLP Architectures	4
3.2 BERT and Bi-Directionality	5
3.3 Knowledge Distillation: Solving the Size Problem	5
3.4 Vector Search & Embeddings	5
4. System Design and Architecture	6
4.1 Microservices Design Pattern	6
4.2 Application Components	6
1. Gradio (The Frontend Service):	6
2. The Backend Service (FastAPI):	6
3. ChromaDB: Database Service	7
4.3 Docker Network Topology	7
5. Implementation Details	8
5.1 Docker Containerization Strategy	8
5.2 Integrating the Sentiment Model	8
6. Results and Evaluation	9
6.1 Performance Metrics	9
6.2 Qualitative Testing	9
7. Discussion and Challenges	10
7.1 Challenge Docker Networking on Windows	10
7.2 Challenge: Dependency Management	10
7.3 Challenge: Memory Constraints	10
8. Conclusion and Future Work	11
8.1 Summary	11
8.2 Future Work	11
9. References	11

Sentiment Analysis with DistilBERT & Chrome DB Integration

1. Abstract

This project involves designing, implementing, and deploying - via containerization - an NLP system that will perform real-time sentiment analysis and store semantic vectors. As large language models go into the wild in this Deep Learning era, much resource consumption and consistency in different environments pose significant challenges. This project tackles such problems by applying Microservices Architecture with Docker and Docker Compose. The kernel logic uses DistilBERT, a knowledge-distilled version of the BERT transformer, to achieve high-accuracy sentiment classification with reduced latency. Furthermore, the system engages ChromaDB, a specialized vector database, to persist semantic embeddings and thus enable future capabilities for Retrieval-Augmented Generation. This will result in a fully reproducible, isolated environment orchestrating a Gradio frontend, FastAPI backend, and vector storage engine that demonstrates mastery in MLOps, NLP pipeline design, and best practices in software engineering.

2. Introduction

2.1 Overview and Motivation

Going from rule-based heuristic systems to statistical models and finally to Deep Learning architectures, the paradigm shift in the field of Artificial Intelligence has happened within the last decade. The Transformer architecture presented by Vaswani et al. in 2017 [1] brought about a paradigm shift in the field of Natural Language Processing. By using the "Self-Attention Mechanism," these Transformers solved the problems in RNNs concerning parallelization and retaining long-term dependencies.

However, there is a critical gap between model development and production deployment. Advanced models such as BERT [2] and GPT-3 [16] have hundreds of millions and even billions of parameters. Inference with these models requires significant computational resources, often necessitating expensive GPUs. Moreover, "Dependency Hell"-the problem where code works on a developer's machine but fails in production because of mismatches in libraries-continues to be a common problem

in industry.

This project tackles these challenges with the following emphases: Efficiency and Reproducibility. By choosing DistilBERT [3], we save 40% in the computational footprint compared to standard BERT. By wrapping the application in Docker containers [12], we ensure that the complex web of dependencies (PyTorch, Transformers, ChromaDB) operates the same way across any host operating system.

2.2 Objectives

The main objectives of the project are:

1. Build an end-to-end Machine Learning pipeline that takes raw text ingested, tokenizes, and does binary sentiment classification accordingly- Positive or Negative.
2. Knowledge Distillation in NLP for CPU-based inference.
- 3.To incorporate the usage of the Vector Database-ChromaDB for storing high-dimensional embeddings to overcome the limitations of conventional relational databases such as SQL, which are not capable of handling semantic data efficiently.
4. To design the solution using Docker Compose by creating an isolated network of interacting services: Frontend, Backend, Database.

3.Literature Review

3.1 Evolution of NLP Architectures

Before 2017, the state-of-the-art in NLP was held by Long Short-Term Memory (LSTM) networks, introduced by Hochreiter and Schmidhuber [6]. Although effective for short sequences, LSTMs process information sequentially, one word at a time. This sequential nature of processing prohibited parallel computing on GPUs and resulted in the "vanishing gradient" problem: the model "forgot" information from the beginning of a long paragraph.

The Transformer Breakthrough: Vaswani et al. of Google Brain came up with the Transformer [1] that entirely dispensed with recurrence. Instead, it relied on an "Attention Mechanism" to weigh the importance of various words in a sentence in

parallel, irrespective of how far they are separated from each other. This enabled massive parallelization and training on datasets larger than ever before.

3.2 BERT and Bi-Directionality

Based on the Transformer, Devlin et al. in 2018 proposed BERT [2]. In contrast to earlier models, which processed text left-to-right (uni-directional), such as OpenAI's GPT-1, BERT was pre-trained on a "Masked Language Model" (MLM) objective. This allowed it to capture context from both directions simultaneously. Using the sentence "I went to the bank to fish," given the context of "fish," BERT recognizes "bank" is referring to a river and not some place to manage your money. BERT established a new state-of-the-art on 11 NLP tasks, including the GLUE benchmark [17].

3.3 Knowledge Distillation: Solving the Size Problem

The issue is that BERT has 110 million parameters, making it slow for real-time applications. Sanh et al. of Hugging Face developed the DistilBERT model [3]. This process is known as Knowledge Distillation by Hinton et al. [19]. A smaller "student" model is trained with the output distribution of the larger "teacher" model-the BERT model.

Table 1: Comparison of different Transformer Models

Feature	BERT-Base [2]	DistilBERT [3]	ROBERTa [18]
Parameters	110 Million	66 Millio	125 Milion
Layers (Depth)	12 Layers	6 Layers	12 Layers
Training Data	Wikipedia+ BooksCorpus	Wikipedia + BooksCorpus	160GB Text Data
Inference Speed	1.0x (Baseline)	-1.6x Faster	Similar to BERT
Performance	100%	97% Of BERT	>100% Of BERT
Hardware Reqs	High CPu Memory	CPU Compatible	High GPU Memory

3.4 Vector Search & Embeddings

Words need to be transformed into vectors in order to use mathematics-based language processing. Early methods such as Word2Vec by Mikolov et al. [4] and

GloVe by Pennington et al. [5] created static embeddings. These failed to capture context. Modern semantic search depends on dynamic embeddings where the vector changes based on sentence structure (Reimers et al., Sentence-BERT [10]).

Vector Databases: ChromaDB Standard SQL databases are incapable of searching efficiently for "similar meanings." Vector Databases employ algorithms such as HNSW (Hierarchical Navigable Small World) [20] to carry out Approximate Nearest Neighbor (ANN) search. In this project, ChromaDB is employed for the persistence of output from the DistilBERT model. This is in accordance with the Retrieval-Augmented Generation (RAG) architecture proposed by Lewis et al. [11], where there is a requirement for a system to "remember" external knowledge.

4. System Design and Architecture

4.1 Microservices Design Pattern

The system follows the pattern of Microservices Architecture described by Bernstein [13]. Other than having a monolithic application where UI and Logic are fused, the system is split into three distinct, loosely coupled containers. That ensures that if the Frontend fails, it will not crash the Backend or the Database. It also makes scaling independently from each other possible; for example, in a high-traffic scenario, we might want to deploy five instances of the Backend container while we keep a single Database container.

4.2 Application Components

1. Gradio (The Frontend Service):

Technology: Python, Gradio Library.

Role: It acts as the client. It generates the HTML/JS interface for the user.

Port: Exposed on 7860.

Logic: Performs input validation, constructs a JSON payload to send to the backend, and handles displaying the response, such as Label and Confidence Score.

2. The Backend Service (FastAPI):

Technology: Python, FastAPI, Uvicorn, PyTorch, Transformers.

Exercising Role: The computational engine. It loads the

distilbert-base-uncased-finetuned-sst-2-english model into memory.

Port: Exposed on 8000.

Logic: Accepts HTTP POST requests. Tokenizes the input string, or in other words, turns text into integers. Passes tensors through the DistilBERT layers. Applies the Softmax function to the output logits to get a probability between 0 and 1. Connects to ChromaDB to log the transaction.

3. ChromaDB: Database Service

Technology: Chroma Vector Store (Docker Image)

Role: persistence layer.

Port: Exposed on 8001 (Internal 8000).

Logic: It will store the document text, UUID, and the metadata being the Sentiment Score for later retrieval.

Table 2: Service Communication Specifications

Source Container	Destination Container	Protocol	Data Format	Purpose
Frontend	Backend	HTTP /REST	JSON	Sending user text for analysis
Backend	Frontend	HTTP / REST	JSON	Returning prediction results
Backend	ChromaDB	HTTP/ RPC	Vector/JSON	Archiving result and metadata

4.3 Docker Network Topology

All three containers are connected to a custom Docker Bridge Network named `ai_network`. This is especially useful because the containers can reach each other by their service names - i.e., `http://backend:8000` - rather than through IP addresses, which change dynamically. This employs Docker's internal DNS resolution capabilities.

5. Implementation Details

5.1 Docker Containerization Strategy

The core of the project is based on Docker for reproducibility. A critical optimization strategy was used in the Dockerfile to handle the large PyTorch library. This is because a standard installation of PyTorch includes CUDA drivers for NVIDIA GPUs, which inflate the image size over 2GB.

Optimizing Technique:

We used the command `pip install torch --index-url`

<https://download.pytorch.org/whl/cpu>. This explicitly asks for the CPU-only version of PyTorch. This brought the size of the image under 500MB, greatly improving the build times and portability. Another optimization we did was use `python:3.9-slim` base image, which is a stripped-down version of Linux (Debian Bullseye) containing only the minimum number of packages required to run Python.

5.2 Integrating the Sentiment Model

We utilized the `distilbert-base-uncased-finetuned-sst-2-english` model available on the Hugging Face Hub [15].

Uncased: The model converts all text to lower case before processing. That greatly reduces the vocabulary size the model needs to learn since "Apple" and "apple" become the same token.

"SST-2": It was fine-tuned on the Stanford Sentiment Treebank dataset [7]. This dataset is superior to standard movie review datasets because it includes labels for phrases within sentences, allowing the model to learn compositional sentiment - how "not" flips the polarity of "good".

5.3 Persistence of Vector Databases

ChromaDB runs as a separate service. The back-end connects using the `chromadb.HttpClient`. Upon analyzing a sentiment, the system generates a UUID [21], and stores the result by using the `collection.add()` method. The logic follows a try-except block to handle the connection failure gracefully. So, if the Database container goes offline, the inference engine (Backend) will keep working without stopping, just skipping the logging step. Therefore, this solution applies the "Circuit Breaker" pattern common for distributed systems reliability engineering.

6. Results and Evaluation

6.1 Performance Metrics

While we did not retrain the model, the pre-trained weights from Hugging Face have established benchmarks on the SST-2 development set [7].

Table 3: Inference performance analysis

Metric	Score/Value	Implication
Accuracy	91.3%	The model correctly predicts Positive/Negative 9 times out of 10.
F1-Score	91.0%	Indicates a balance between Precision and Recall.
CPU Latency	-150ms	Average time to process one sentence on an Intel Core i5.
Memory Usage	268 MB	The RAM required to hold the model weights
Disk Usage	450 MB	The size of the Docker Image (compressed).

6.2 Qualitative Testing

The system was exposed to a "Stress Test" with complex linguistic structures in order to assess robustness beyond simple keywords.

1. Test Case 1: Simple Polarity

Input: "I absolutely love this new feature."

Result: POSITIVE (Score: 0.9998).

Analysis: The model identifies the strong emotive word "love".

2. Test Case 2: Handling of Negation

Input: "I do not dislike the movie."

Outcome: POSITIVE, with a score of 0.85.

Analysis: This case is difficult. The words "not" and "dislike" appear as negative words, but when combined, they become a double negative. The model correctly identified the positive sentiment, validating the attention mechanism's ability to capture context.

3. Test Case 3: Adversarial/Sarcastic

Input: "Oh great, my car broke down again."

Result: NEGATIVE (Score: 0.99).

Analysis: The word "great" is normally positive, but the model used the context "broke down" to classify it as negative/sarcastic correctly.

7. Discussion and Challenges

7.1 Challenge Docker Networking on Windows

The Docker build on Windows 10/11 presented a substantial technical challenge. It repeatedly failed to build due to an error: dial tcp [.] connectex: A connection attempt failed. This is a known issue with Docker Desktop and how it handles IPv6 DNS resolution.

The resolution involved explicitly setting the Docker Daemon DNS configuration to Google's public DNS (8.8.8.8, 8.8.4.4) as well as disabling IPv6 on the host network adapter. This then forced the container to use IPv4 when sending outgoing requests to pypi.org.

7.2 Challenge: Dependency Management

A version conflict arose during the integration of the Frontend. The gradio library depends on the functionality of the huggingface_hub library. In a recent update, the HfFolder class was removed in the huggingface_hub library, due to which the container crashed with an ImportError.

Solution: We used "Dependency Pinning" in requirements.txt, namely specifying huggingface-hub==0.24.6. That makes the environment deterministic - it will behave exactly the same if deployed in 5 years.

7.3 Challenge: Memory Constraints

PyTorch is resource-intensive. Initial attempts to build the container crashed because Docker on Windows defaults to 2GB of memory allocation.

Solution: We optimized the build by removing the cache files with --no-cache-dir and informed the user to please set up Docker Desktop to use 4GB RAM in order to handle the model loading phase.

8. Conclusion and Future Work

8.1 Summary

This project successfully demonstrates the deployment of a modern Deep Learning application using containerization. By coupling the efficiency of DistilBERT [3] with the portability provided by Docker [12], we developed a solid sentiment analyzer that can cope with requirements imposed by modern DevOps. The system isn't just a passive processor because ChromaDB will integrate, turning it into an active data store, fully capable of retaining semantic history. In sum, this delivers against the requirements set forth in this assignment: implementation of ML pipeline, use of Deep Learning, integration of Vector DB, and complete Dockerization.

8.2 Future Work

GPU Acceleration: The current system runs on the CPU. Future work covers setting up the nvidia-container-toolkit so that Docker can reach the host GPU, CUDA, which would lead to a reduction of inference latency from 150ms to <20ms [9]. **Semantic Search UI:** ChromaDB stores the data, but the current UI only displays sentiment. A new tab can be added on Gradio that queries the database; for example, "Find similar comments to this," using Cosine Similarity. **Authentication:** Adding JWT to the FastAPI backend so that API endpoints are protected from unauthorized use.

9. References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019.
- [3] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of

- BERT: smaller, faster, cheaper and lighter," arXiv preprint arXiv:1910.01108, 2019.
- [4] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," arXiv preprint arXiv:1301.3781, 2013.
 - [5] J. Pennington, R. Socher, and C. Manning, "GloVe: Global vectors for word representation," in Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2014.
 - [6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.
 - [7] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Y. Ng, and C. Potts, "Recursive deep models for semantic compositionality over a sentiment treebank," in Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2013.
 - [8] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, no. 7553, pp. 436–444, 2015.
 - [9] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535–547, 2019.
 - [10] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2019.
 - [11] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), 2020.
 - [12] D. Merkel, "Docker: lightweight linux containers for consistent development and deployment," Linux Journal, vol. 2014, no. 239, 2014.
 - [13] D. Bernstein, "Containers and cloud: From LXC to Docker to Kubernetes," IEEE Cloud Computing, vol. 1, no. 3, pp. 81–84, 2014.
 - [14] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2016.
 - [15] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 2020.
 - [16] T. Brown et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

- [17] A. Wang et al., "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in Proceedings of the International Conference on Learning Representations (ICLR), 2019.
- [18] Y. Liu et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv preprint arXiv:1907.11692, 2019.
- [19] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv preprint arXiv:1503.02531, 2015.
- [20] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 42, no. 4, pp. 824-836, 2018.
- [21] P. Leach, M. Mealling, and R. Salz, "A Universally Unique IDentifier (UUID) URN Namespace," IETF RFC 4122, 2005.